

Lecture Notes: ETL With PySpark

Course Overview

Key Definitions

- **FastAPI:** A modern, high-performance Python web framework for building APIs and microservices with automatic documentation and async support.
- **Docker:** A containerisation platform that packages code and dependencies into lightweight, portable units called containers.
- **Containerisation:** The process of bundling software with its environment to ensure consistent execution across different systems.

Why Learn FastAPI and Docker?

Data engineers and developers often need to make models and scripts accessible to other systems or teams. FastAPI enables you to create APIs that expose your application logic, while Docker ensures that your application runs reliably in any environment. Together, they bridge the gap between **development** and **deployment**.

By the end of this module, you will be able to:

- Build RESTful APIs using FastAPI.
- Deploy Python scripts as API endpoints.
- Understand Docker architecture and its core components.
- Containerise a FastAPI application for portable and scalable deployment.
- Push and run containers in any environment (local or cloud).

FastAPI

Key Concepts

- FastAPI simplifies API development using Python, leveraging type hints and asynchronous programming.
- It automatically generates **OpenAPI documentation** and **interactive Swagger UI** for testing.

Features of FastAPI

- **High Performance:** Built on ASGI and uses asynchronous requests.

- **Automatic Validation:** Uses Python type hints and Pydantic models.
- **Built-in Docs:** Provides Swagger UI and ReDoc interfaces.
- **Ease of Integration:** Works seamlessly with databases, ML models, and background tasks.

Core Components

- **Path Parameters:** Used to capture variables directly from URL routes.
- **Query Parameters:** Pass additional filters or options in requests.
- **Request Body:** Data sent in POST/PUT methods, validated using Pydantic models.
- **Response Models:** Define consistent response structures.

Example: Simple FastAPI Endpoint

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Welcome to FastAPI"}

@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id, "status": "Fetched successfully"}
```

Running FastAPI

- Save your script as app.py
- Run the app with:
- `uvicorn app:app --reload`
- Visit <http://127.0.0.1:8000/docs> for interactive API documentation.

Use Cases in Data Applications

- Triggering ETL jobs via API calls.
- Exposing machine learning predictions.
- Building microservices that communicate through APIs.

Designing APIs for Data Applications

- APIs act as a bridge between frontend systems and backend logic (e.g., data transformation scripts or ML models).
- A good API design ensures reusability, scalability, and security.

Key Practices

- **Modular Code:** Separate logic into different files (routers, models, main app).
- **Validation:** Use Pydantic for enforcing input formats.
- **Error Handling:** Implement proper exception responses.
- **Versioning:** Maintain /v1, /v2 endpoints for backward compatibility.

Example: Adding Data Validation

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float
    in_stock: bool

@app.post("/add-item")
def add_item(item: Item):
    return {"message": f"Item {item.name} added successfully"}
```

Docker

Key Concepts

Docker enables developers to package applications along with all dependencies into containers. These containers ensure consistent performance across systems.

Why Docker?

- Eliminates “works on my machine” issues.
- Simplifies deployment and scaling.
- Lightweight and faster than virtual machines.

Docker Architecture

- **Dockerfile:** Instructions to build an image.
- **Image:** A read-only template used to create containers.
- **Container:** A running instance of an image.
- **Docker Engine:** The runtime that builds and manages containers.
- **Docker Hub:** Cloud-based registry for sharing images.

Docker Workflow

1. Write a **Dockerfile** describing your app environment.
2. Build an **image** from the Dockerfile.
3. Run a **container** from the image.

Basic Commands

```
docker build -t myapp .
```

```
docker run -p 8000:8000 myapp
```

```
docker ps
```

```
docker stop <container_id>
```

```
docker push <username>/myapp
```

Containerising FastAPI with Docker

Goal: Package the FastAPI application in a Docker container so it can run anywhere.

Step 1: Create a Dockerfile

Base image

FROM python:3.10-slim

Set working directory

WORKDIR /app

Copy files

COPY . /app

Install dependencies

RUN pip install fastapi uvicorn

Expose port

EXPOSE 8000

Command to run the app

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]

Step 2: Build and Run the Container

docker build -t fastapi_app .

docker run -d -p 8000:8000 fastapi_app

Step 3: Test the Application

Visit:

<http://localhost:8000> → to see the app

<http://localhost:8000/docs> → for Swagger UI

Benefits of Containerisation

- **Portability:** Run on any OS or cloud platform.
- **Scalability:** Deploy multiple containers with load balancers.
- **Consistency:** Identical runtime environments.

- **Version Control:** Manage app versions through Docker images.

Ask an LLM

1. How does FastAPI differ from traditional frameworks like Flask or Django?
2. Why is Docker preferred over virtual machines in modern DevOps workflows?
3. What are the advantages of using FastAPI with asynchronous endpoints?
4. Explain the role of the Dockerfile in application deployment.
5. How would you deploy a FastAPI Docker container on a cloud platform like AWS or Azure?

Summary

- **FastAPI** enables high-performance, async-based API development with automatic documentation.
- **Docker** ensures your applications run consistently across environments.
- Together, they streamline the transition from **local development** → **production deployment**.
- FastAPI allows building microservices; Docker makes them portable, scalable, and reliable.

Topics Commonly Asked in FastAPI & Docker Interviews

1. Difference between **Flask** and **FastAPI**.
2. How FastAPI uses **type hints** and **Pydantic models** for validation.
3. Difference between **GET, POST, PUT, DELETE** methods in REST APIs.
4. Steps to **containerise** a FastAPI app.
5. Explain **Docker image vs container**.
6. How to manage environment variables and secrets in Docker.
7. Best practices for Dockerfile optimisation.
8. Deploying Docker containers in **AWS ECS, Azure Container Apps**, or **Kubernetes**.
9. Importance of **asynchronous requests** in FastAPI for performance.

10. Common **Docker networking concepts** (bridge, host, overlay).